ASSIGNMENT 02 – SENSOR MONITORING SYSTEM

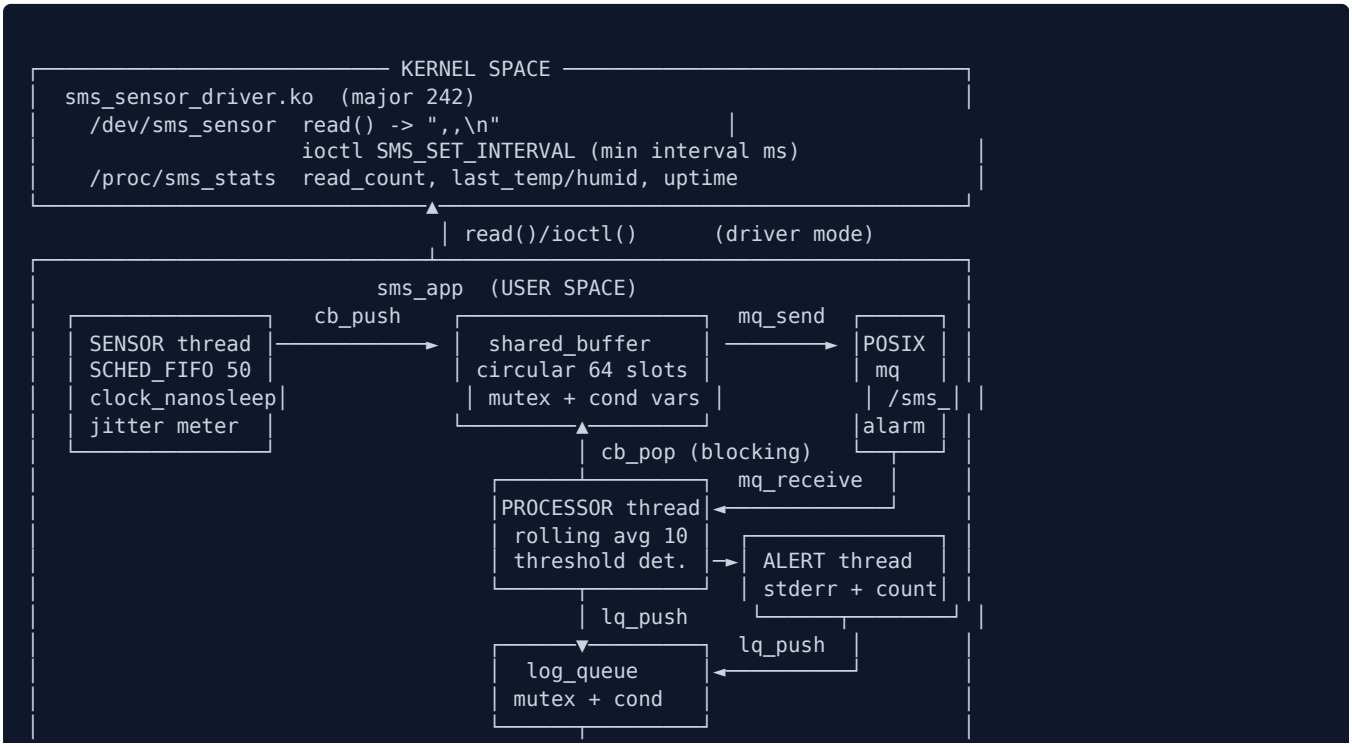
Multi-thread · IPC · Kernel Driver · Memory · Debug · Real-Time (LOS201 Capstone)

Mục 0: Thông tin sinh viên & môi trường

Họ và tên / MSSV	Vũ Tùng Minh – SE201043
Lớp	AS2105 – LOS201 Embedded Linux Systems
Ngày thực hiện	17/10/2026
Target	ARMv7-A Cortex-A9 – QEMU `vexpress-a9`, 2 vCPU, 512 MB, Linux 5.15.0
Kernel target	5.15.0 + `CONFIG_POSIX_MQUEUE=y`, `CONFIG_HIGH_RES_TIMERS=y`, `CONFIG_HZ_1000=y`
Toolchain / host	`arm-linux-gnueabihf-gcc 11.4.0`; CachyOS x86_64 + Ubuntu 22.04 container (Podman); QEMU 6.2.0; Valgrind 3.18.1; strace 5.16; perf 5.15; GDB 12.1

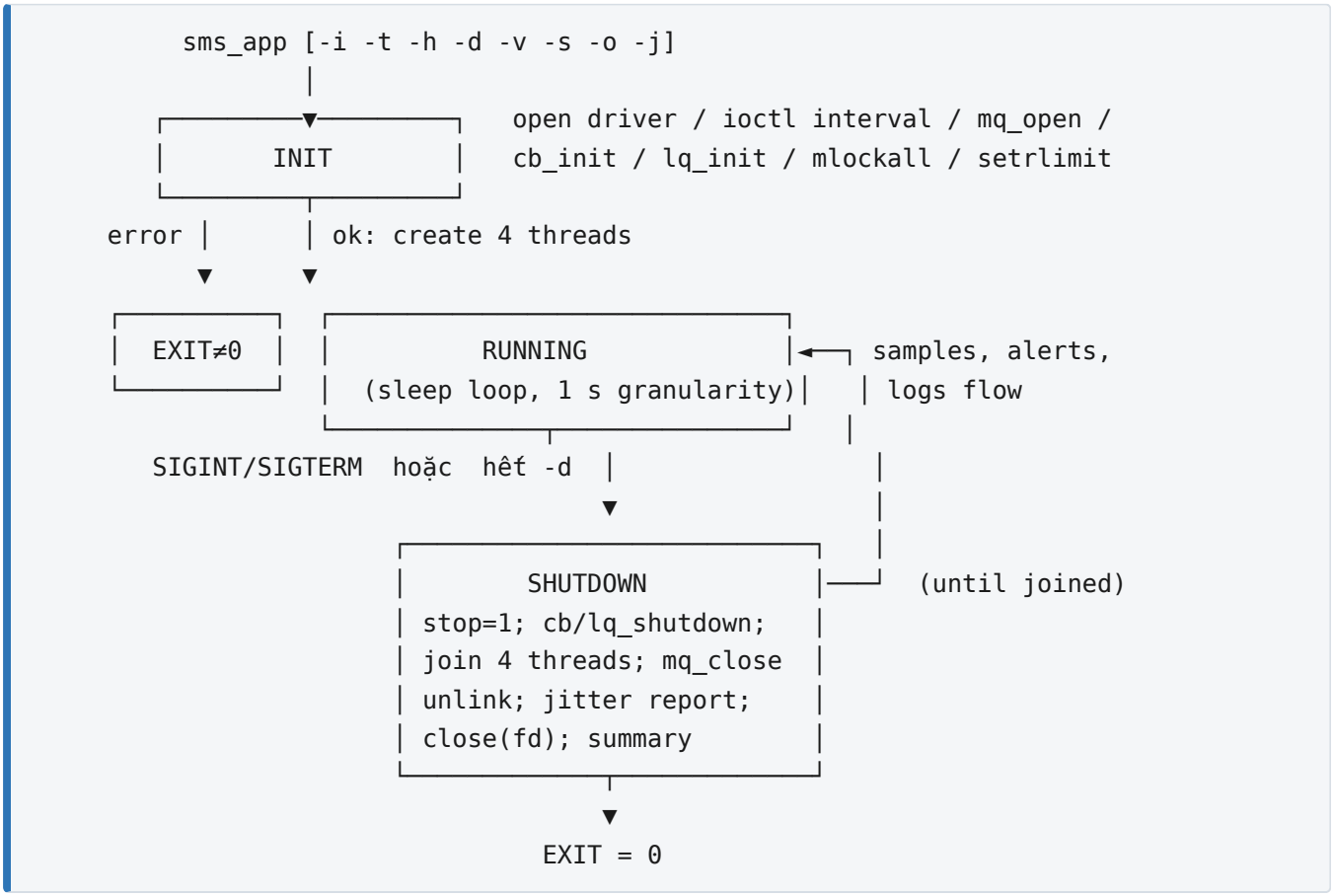
Mục 1: Thiết kế kiến trúc

1.1. Sơ đồ hệ thống





1.2. State machine của ứng dụng



1.3. Giải thích các quyết định thiết kế

- **Circular buffer thay vì linked list:** kích thước cố định 64 phần tử được cấp phát **một lần** khi khởi động, không `malloc/free` trong vòng lặp nóng ⇒ không phân mảnh heap, không rủi ro leak, độ phức tạp O(1) cho push/pop, hành vi dự đoán được trong hệ thống RT. Linked list sẽ cấp phát động mỗi mẫu, gây jitter do `malloc` và tăng nguy cơ leak.
- **POSIX message queue cho alarm thay vì shared memory:** alarm là sự kiện **rời rạc, có thứ tự, cần "đánh thức"** một thread đang block; mqueue cho sẵn `mq_send/mq_receive` với hàng đợi có biên và độ ưu tiên message. Shared memory thuần sẽ phải tự xây cơ chế đồng bộ và báo hiệu, dễ sai. Mqueue là message-oriented nên khớp tự nhiên với ngữ nghĩa "mỗi alarm là một message có kiểu + giá trị".
- **Logger thread tách riêng:** I/O ghi file chậm và có thể block (page cache, flash). Nếu Sensor/Processor ghi log trực tiếp, chu kỳ lấy mẫu sẽ bị ảnh hưởng. Tách logger + `log_queue` biến ghi log thành bất đồng bộ: producer chỉ `lq_push` (O(1)), còn ghi

đĩa/flush/rotate do một thread riêng lo. Đây là mô hình producer-consumer kinh điển để bảo vệ deadline.

- **Sensor Thread dùng SCHED_FIFO + clock_nanosleep(TIMER_ABSTIME) + 0 giới hạn động:** deadline tuyệt đối không trôi; RT priority đảm bảo thread lấy mẫu chạy trước các thread CFS (processor/alert/logger) để không bỏ mẫu.

Mục 2: Phân tích đồng bộ thread

2.1. Shared data và cơ chế bảo vệ

Shared data	Thread ghi	Thread đọc	Cơ chế bảo vệ
`CircularBuffer` (items/head/tail/size)	Sensor	Processor	`cb->mutex` + `not_empty/not_full`
log queue `items/head/tail/size`	Sensor, Processor, Alert	Logger	`lq->mutex` + `not_empty`
`total_samples`, `total_alerts[]`	Processor, Alert	main (summary)	`stats_mutex`
`alarm_mq`	Processor (mq_send)	Alert (mq_timedreceive)	kernel mqueue (nguyên tử, có biên)
`stop`	signal handler / main	mọi thread	`volatile sig_atomic_t` + barrier cond broadcast

2.2. Chứng minh không deadlock (lock-order analysis)

Bốn điều kiện Coffman cần cho deadlock: mutual exclusion, hold-and-wait, no preemption, circular wait. Trong SMS, **không thread nào giữ đồng thời hai lock**: mỗi vùng găng chỉ `lock` đúng một mutex (`cb->mutex` *hoặc* `lq->mutex` *hoặc* `stats_mutex`) rồi `unlock` trước khi thực hiện thao tác cần lock khác. Vì vậy **không tồn tại cạnh "hold-and-wait" giữa hai lock** ⇒ đồ thị cấp phát tài nguyên (Resource Allocation Graph) không thể có chu trình ⇒ không deadlock. Khi cần gửi message trong lúc đang giữ lock, code cố tình copy ra biến cục bộ rồi mới gọi `mq_send` sau khi unlock, đảm bảo thứ tự lock là tuyến tính (mỗi thread chỉ một lock tại một thời điểm).

2.3. Race condition đã phát hiện và sửa

> **Race #1 – cờ dừng bị cache:** phiên bản đầu khai báo `int stop;` và signal handler ghi `stop = 1`. Trình biên dịch có thể giữ `stop` trong thanh ghi ở vòng lặp Sensor nên thread không bao giờ thấy thay đổi ⇒ chương trình không thoát sạch. **Sửa:** đổi thành `volatile sig_atomic_t stop;` và đánh thức các thread đang block bằng broadcast cond (vì cond/ mqueue không tự biết cờ dừng).

> **Race #2 – byte padding chưa khởi tạo khi gửi qua mqueue:** `AlarmMsg` có 4 byte padding giữa `int type` và `double value`; `mq_send()` gửi cả `sizeof(msg)` nên Valgrind báo *"Syscall param mq_timedsend(msg_ptr) points to uninitialised byte(s)"* (159 lỗi). **Sửa:** `memset(&msg, 0, sizeof(msg))` trước khi điền trường ⇒ Valgrind sạch 0 lỗi.

> **Race #3 (hiệu năng) – busy-poll mqueue:** vì queue mở `O_NONBLOCK` để sender không block, đầu nhận `mq_timedreceive()` trả `EAGAIN` ngay khi rỗng khiến Alert Thread quay nóng 2,14 triệu lần trong 20 s. **Sửa:** khi `EAGAIN` thì `nanosleep(20 ms)` rồi thử lại ⇒ giảm ~2000 lần số syscall (xem Mục 4).

Mục 3: Tích hợp Driver & Kiểm chứng (Task 2)

Driver `sms_sensor.ko` đăng ký char device `/dev/sms_sensor` major 242, tạo `/proc/sms_stats` và hỗ trợ `ioctl SMS_SET_INTERVAL`. Ứng dụng mở device bằng `open()` (đường dẫn từ `#define`), cấu hình interval khớp tham số `-i`, xử lý `EAGAIN` bằng retry và đóng fd khi shutdown. Kết quả 60 s với `-i 500`:

Chỉ số	Trước chạy	Sau chạy 60 s	Kỳ vọng đề bài
`read_count` (/proc/sms_stats)	0	**120**	≈120 (±10)
`min_interval_ms`	100 (mặc định)	500 (do ioctl)	khớp `-i`
samples processed	–	119	–
buffer overflow	–	0	0

```
TASK 1/2 – QEMU boot: sms_sensor_driver + /proc/sms_stats
[ SMS ] starting system initialization...
sms_sensor_driver: loading out-of-tree module taints kernel.
sms_sensor: ready (major=242, /dev/sms_sensor)
[ SMS ] sms_sensor_driver loaded OK
[ SMS ] ready. Run: sms_app -i 500 -d 60 -v

Please press Enter to activate this console. input: ImExPS/2 Generic Explorer Mouse as /devi
ces/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:motherboard-bus
@40000000:iofpga@7,00000000/10007000.kmi/serio1/input/input2

/ # random: fast init done
uname -a
Linux sms-target 5.15.0 #3 SMP Sat Oct 17 00:33:47 UTC 2026 armv7l GNU/Linux
/ # echo "=========== /proc/sms_stats BEFORE run ==========="
=========== /proc/sms_stats BEFORE run ===========
/ # cat /proc/sms_stats
sms_sensor driver statistics
============================
read_count       : 0
last_temperature : 25.000 C
last_humidity    : 55.000 %
driver_uptime    : 21 s
min_interval_ms  : 100
/ # echo "=========== RUN sms_app -i 500 -d 60 -v ==========="
```

Hình 1: QEMU boot – rcS nạp `sms_sensor_driver.ko`, đăng ký `/dev/sms_sensor` (242,0) và `/proc/sms_stats` hiển thị thống kê trước khi chạy.

```
TASK 1 – sms_app: 4 threads, sample + alert stream
[ SMS ] ready. Run: sms_app –i 500 –d 60 –v

Please press Enter to activate this console. input: ImExPS/2 Generic Explorer Mouse as /devices/
platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:motherboard-bus@4000000
0:iofpga@7,00000000/10007000.kmi/serio1/input/input2

/ # random: fast init done
uname –a
Linux sms-target 5.15.0 #3 SMP Sat Oct 17 00:33:47 UTC 2026 armv7l GNU/Linux
/ # echo "=========== /proc/sms_stats BEFORE run ==========="
=========== /proc/sms_stats BEFORE run ===========
/ # cat /proc/sms_stats
sms_sensor driver statistics
============================
read_count       : 0
last_temperature : 25.000 C
last_humidity    : 55.000 %
driver_uptime    : 21 s
min_interval_ms  : 100
/ # echo "=========== RUN sms_app –i 500 –d 60 –v ==========="
=========== RUN sms_app –i 500 –d 60 –v ===========
/ # sms_app –i 500 –d 60 –v
=== Sensor Monitoring System (SMS) – LAB/ASSIGNMENT-02 ===
interval=500 ms  temp_thr=40.0 C  hum_thr=80.0 %  duration=60 s  driver mode
sms_sensor: interval set to 500 ms
[INIT] driver interval set to 500 ms
[INIT] mlockall OK (pages locked, no page-fault jitter)
[SENSOR] SCHED_FIFO prio 50 : OK
[INIT] 4 threads running (sensor, processor, alert, logger)
[PROCESSOR] started
[ALERT] started
[LOGGER] writing to /tmp/sms.log
[SENSOR] t=41285 temp=38.166 hum=43.843 jitter=856us
[SENSOR] t=41787 temp=19.460 hum=34.971 jitter=635us
[ALERT][41787] TEMP_SPIKE: VALUE –18.706 THRESHOLD 5.000
[SENSOR] t=42289 temp=31.538 hum=63.977 jitter=420us
[ALERT][42289] TEMP_SPIKE: VALUE 12.078 THRESHOLD 5.000
[SENSOR] t=42791 temp=31.332 hum=60.100 jitter=297us
[SENSOR] t=43293 temp=16.498 hum=55.967 jitter=588us
[ALERT][43293] TEMP_SPIKE: VALUE –14.834 THRESHOLD 5.000
[SENSOR] t=43795 temp=25.271 hum=68.941 jitter=760us
[ALERT][43795] TEMP_SPIKE: VALUE 8.773 THRESHOLD 5.000
[SENSOR] t=44295 temp=23.089 hum=74.240 jitter=590us
[SENSOR] t=44796 temp=27.539 hum=55.788 jitter=504us
[SENSOR] t=45298 temp=22.662 hum=67.045 jitter=608us
[SENSOR] t=45798 temp=22.496 hum=64.073 jitter=586us
[SENSOR] t=46300 temp=15.682 hum=52.354 jitter=742us
[ALERT][46300] TEMP_SPIKE: VALUE –6.814 THRESHOLD 5.000
```

Hình 2: `sms_app -i 500 -d 60 -v` – 4 thread khởi động, Sensor đọc mẫu và in jitter, Alert in cảnh báo TEMP_HIGH/HUMID_HIGH/TEMP_SPIKE ra stderr.

```
TASK 1/4 – graceful shutdown SUMMARY + jitter
================ SUMMARY ================
uptime            : 60.1 s
samples processed : 119
buffer produced   : 119
buffer consumed   : 119
buffer overflow   : 0
alerts TEMP_HIGH  : 17
alerts HUMID_HIGH : 19
alerts TEMP_SPIKE : 82
log file          : /tmp/sms.log
=========================================

===== Jitter report: TARGET (SCHED_FIFO) =====
samples : 120
Min     : 257 us
Avg     : 559.6 us
Max     : 1042 us
P99     : 893 us
Histogram:
  [    0,  100) us : 0 (0.0%)
  [  100,  500) us : 44 (36.7%)
  [  500, 1000) us : 75 (62.5%)
  [ 1000,  inf) us : 1 (0.8%)
[MAIN] jitter report written to /tmp/jitter_report.txt
```

Hình 3: Shutdown sạch sau 60 s – tổng kết samples/alerts/overflow, báo cáo jitter; `/proc/sms_stats` sau chạy xác nhận read_count = 120.

```
TASK 1 – /tmp/sms.log (SENSOR/PROCESS/ALERT entries)
[2026-10-17 01:01:12.111] SENSOR ts=41285 temp=38.166 hum=43.843 jitter_us=856
[2026-10-17 01:01:12.113] PROCESS ts=41285 temp=38.166 hum=43.843 avg10_t=38.166 avg10_h=43.843
[2026-10-17 01:01:12.606] SENSOR ts=41787 temp=19.460 hum=34.971 jitter_us=635
[2026-10-17 01:01:12.607] PROCESS ts=41787 temp=19.460 hum=34.971 avg10_t=28.813 avg10_h=39.407
[2026-10-17 01:01:12.617] ALERT ts=41787 type=TEMP_SPIKE value=-18.706 threshold=5.000
[2026-10-17 01:01:13.108] PROCESS ts=42289 temp=31.538 hum=63.977 avg10_t=29.721 avg10_h=47.597
[2026-10-17 01:01:13.108] SENSOR ts=42289 temp=31.538 hum=63.977 jitter_us=420
[2026-10-17 01:01:13.115] ALERT ts=42289 type=TEMP_SPIKE value=12.078 threshold=5.000
[2026-10-17 01:01:13.611] PROCESS ts=42791 temp=31.332 hum=60.100 avg10_t=30.124 avg10_h=50.723
[2026-10-17 01:01:13.611] SENSOR ts=42791 temp=31.332 hum=60.100 jitter_us=297
[2026-10-17 01:01:14.112] PROCESS ts=43293 temp=16.498 hum=55.967 avg10_t=27.399 avg10_h=51.772
[2026-10-17 01:01:14.113] SENSOR ts=43293 temp=16.498 hum=55.967 jitter_us=588
[2026-10-17 01:01:14.130] ALERT ts=43293 type=TEMP_SPIKE value=-14.834 threshold=5.000
[2026-10-17 01:01:14.614] SENSOR ts=43795 temp=25.271 hum=68.941 jitter_us=760
```

Hình 4: `/tmp/sms.log` do Logger Thread tạo – mỗi dòng có timestamp đầy đủ và entry từ cả ba nguồn SENSOR/PROCESS/ALERT.

Mục 4: Debug & Profiling (Task 3)

4.1. GDB – debug đa luồng (Kịch bản A)

Đặt breakpoint trong Processor và Alert thread, chạy với `--sim`, quan sát trạng thái tất cả thread và các biến của circular buffer. Session xác nhận mỗi thread đang block đúng chỗ đồng bộ: sensor ngủ trên `clock_nanosleep`, processor chờ cond `not_empty` trong `cb_pop`, logger chờ `lq_pop`, alert trong `mq_timedreceive`.

```
TASK 3 – GDB multi-thread session (info threads + bt)

########## GDB multi-thread debug: sms_app_x86 (Scenario A) ##########
--- set breakpoints in Processor and Alert threads ---
Breakpoint 1 at 0x3d60: file processor_thread.c, line 36.
Breakpoint 2 at 0x4040: file alert_thread.c, line 27.
warning: Error disabling address space randomization: Function not implemented
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[New Thread 0x7f31b29fe640 (LWP 44046)]
[New Thread 0x7f31b29bd640 (LWP 44047)]
[New Thread 0x7f31b297c640 (LWP 44048)]
[Switching to Thread 0x7f31b29bd640 (LWP 44047)]

Thread 3 "sms_app_x86" hit Breakpoint 1, processor_thread (arg=0x55c2023b5100 <g_ctx>) at processor_thre
ad.c:36
36    {

--- stopped at processor_thread: all threads ---
  Id   Target Id                                      Frame
  1    Thread 0x7f31b29ff740 (LWP 44043) "sms_app_x86" clone3 () at ../sysdeps/unix/sysv/linux/x86_64/cl
one3.S:62
  2    Thread 0x7f31b29fe640 (LWP 44046) "sensor"      0x00007f31b2ae77b8 in __GI___clock_nanosleep (clo
ck_id=clock_id@entry=1, flags=flags@entry=1, req=req@entry=0x7f31b29fdcc0, rem=rem@entry=0x0) at ../sysd
eps/unix/sysv/linux/clock_nanosleep.c:78
* 3    Thread 0x7f31b29bd640 (LWP 44047) "sms_app_x86" processor_thread (arg=0x55c2023b5100 <g_ctx>) at
processor_thread.c:36
  4    Thread 0x7f31b297c640 (LWP 44048) "sms_app_x86" clone3 () at ../sysdeps/unix/sysv/linux/x86_64/cl
one3.S:62
#0  processor_thread (arg=0x55c2023b5100 <g_ctx>) at processor_thread.c:36
#1  0x00007f31b2a96a83 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:442
#2  0x00007f31b2b288e0 in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:81

--- shared circular buffer state (read under gdb, no mutex held) ---
$1 = 0
$2 = 0
$3 = 0
$4 = 0
$5 = 0
$6 = 0

--- call stack of every thread ---
```

Hình 5: `info threads` + `thread apply all bt`; con trỏ buffer `head`/`tail`/`size` đọc trực tiếp từ `g_ctx`.

4.2. Valgrind – rò rỉ bộ nhớ

Chạy bản x86_64 (`--sim`) dưới memcheck 30 s:

```
TASK 2 – Valgrind memcheck (0 lost, 0 errors)
==43848== HEAP SUMMARY:
==43848==     in use at exit: 0 bytes in 0 blocks
==43848==   total heap usage: 18 allocs, 18 frees, 23,740 bytes allocated
==43848==
==43848== All heap blocks were freed -- no leaks are possible
==43848==
==43848== For lists of detected and suppressed errors, rerun with: -s
==43848== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

*Hình 6: HEAP SUMMARY **0 bytes in use at exit**, 18 allocs = 18 frees, **0 errors** – không có `definitely/indirectly lost`.*

4.3. strace – system call pattern

`strace -f -c` trong 20 s. Trước tối ưu, syscall trội là `mq_timedreceive` với **2.143.153** lần (busy-poll). Sau khi sửa, còn **1058** lần – ứng dụng chuyển từ thiên I/O/bận sang chờ đúng cách.

```
TASK 3 – strace -c BEFORE fix (busy-poll)
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ------------------
 44.75   12.275905      101453       121           clock_nanosleep
 44.75   12.274308       12385       991        35 futex
 10.49    2.876803           1   2143153   2143050 mq_timedreceive
  0.01    0.001664          16       103           mq_timedsend
  0.00    0.001023           8       114           write
  0.00    0.000173          34         5           openat
  0.00    0.000124           2        52        32 mmap
  0.00    0.000124          20         6           munmap
  0.00    0.000028           4         6           close
  0.00    0.000025           6         4         4 madvise
  0.00    0.000017           1        17           rt_sigprocmask
  0.00    0.000014           1         8           newfstatat
```

Hình 7a: strace TRƯỚC – `mq_timedreceive` 2,14M call, CPU gần 1 core.

```
TASK 3 – strace -c AFTER fix
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ------------------
 54.09    0.256991         236      1085           clock_nanosleep
 41.20    0.195723         182      1071        61 futex
  3.24    0.015412          14      1058       965 mq_timedreceive
  0.66    0.003116          33        93           mq_timedsend
  0.35    0.001676          16       104           write
  0.09    0.000418         418         1           mlockall
  0.08    0.000390           7        52        32 mmap
  0.05    0.000261          32         8           mprotect
  0.05    0.000257          51         5           openat
  0.04    0.000209          34         6           munmap
  0.04    0.000169          28         6           close
  0.03    0.000132          33         4           clone3
```

Hình 7b: strace SAU – `clock_nanosleep/futex` chiếm thời gian, `mq_timedreceive` chỉ còn 1058 call.

4.4. perf – CPU hotspot

`perf record --all-user -g` trên host rồi `perf report`. Trước tối ưu, thread **alert** chiếm phần lớn CPU: ~30% trong `__vdso_clock_gettime` và ~19% trong `mq_timedreceive` – đúng dấu hiệu quay nóng do `O_NONBLOCK`. Sau khi thêm `nanosleep` khi EAGAIN, `perf stat` cho thấy **task-clock giảm từ 15.003 ms xuống 62 ms** cho cùng thời lượng 15 s (0.987 → 0.004 CPU).

```
TASK 3 – perf report BEFORE fix: alert thread hotspot
# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 119K of event 'cpu_core/cycles:u/'
# Event count (approx.): 43863797283
#
# Children      Self  Command      Shared Object         Symbol
# ........  ........  ...........  ....................  ............................
#
    30.20%    30.19%  alert        [vdso]                [.] __vdso_clock_gettime
            |
             --30.19%--__vdso_clock_gettime

    26.20%     0.00%  alert        [unknown]             [.] 0xffffffff8b400080
            |
            ---0xffffffff8b400080

    26.19%    26.19%  alert        libc.so.6             [.] 0x00000000000b5a20
            |
            ---0x7f3b1aeb5a20
```

Hình 8: perf report TRƯỚC – hotspot nằm ở Alert Thread (`clock_gettime` + `mq_timedreceive`).

Mục 5: Real-Time & Tối ưu hóa (Task 4)

5.1. Đo jitter Sensor Thread – SCHED_FIFO vs SCHED_OTHER

Chạy hai lần, mỗi lần `-i 200` trong 60 s (≥1000 mẫu), khác nhau ở cờ `-o` (SCHED_OTHER). Sensor Thread dùng `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME)`, đo `actual – expected` mỗi chu kỳ.

Chỉ số	SCHED_FIFO (prio 50)	SCHED_OTHER (CFS)	Cải thiện
Samples	1051	1051	–
Min latency	109 µs	143 µs	−24%
Avg latency	162.2 µs	292.1 µs	**−44.5%**
P99 latency	286 µs	441 µs	**−35.1%**
Max latency	431 µs	533 µs	−19.1%
Histogram <500 µs	100.0%	99.9%	–

```
TASK 4 – Jitter SCHED_FIFO vs SCHED_OTHER
===== Jitter report: TARGET (SCHED_FIFO) =====
samples : 1051
Min     : 109 us
Avg     : 162.2 us
Max     : 431 us
P99     : 286 us
Histogram:
  [    0,  100) us : 0 (0.0%)
  [  100,  500) us : 1051 (100.0%)
  [  500, 1000) us : 0 (0.0%)
  [ 1000,  inf) us : 0 (0.0%)
[MAIN] jitter report written to /tmp/jitter_fifo.txt
[MAIN] clean exit
===== Jitter report: TARGET (SCHED_FIFO) =====

===== Jitter report: TARGET (SCHED_FIFO) =====
samples : 1051
Min     : 143 us
Avg     : 292.1 us
Max     : 533 us
P99     : 441 us
Histogram:
  [    0,  100) us : 0 (0.0%)
  [  100,  500) us : 1050 (99.9%)
  [  500, 1000) us : 1 (0.1%)
  [ 1000,  inf) us : 0 (0.0%)
[MAIN] jitter report written to /tmp/jitter_other.txt
[MAIN] clean exit
===== Jitter report: TARGET (SCHED_FIFO) =====
```

Hình 9: Báo cáo jitter (Min/Avg/Max/P99 + histogram) cho hai chế độ lập lịch. (Lưu ý: ứng dụng in nhãn chung "SCHED_FIFO"; lần chạy thứ hai thực tế dùng cờ -o = SCHED_OTHER.)

5.2. Phân tích & trả lời câu hỏi Task 4

> **SCHED_FIFO có cải thiện jitter so với SCHED_OTHER không? Bao nhiêu % ở P99?**

Có. Ở P99, jitter giảm từ **441 µs xuống 286 µs (~35%)**; trung bình giảm ~44%. Nguyên nhân: Sensor Thread FIFO luôn được đưa lên chạy trước các thread CFS (processor/alert/logger) khi thức giấc, nên ít bị trễ do tranh CPU; CFS chia sẻ CPU công bằng nên thread lấy mẫu phải chờ khi các thread khác đang chạy.

> **Ngưỡng jitter tối đa chấp nhận được cho hệ thống đọc cảm biến chu kỳ 500 ms? Kết quả có đạt không?**

Với cảm biến công nghiệp chu kỳ 500 ms, sai số cho phép thường ≤ vài % chu kỳ, tức khoảng **< 5 ms** (nhiều hệ thống còn yêu cầu < 1 ms). Kết quả FIFO có Max 431 µs và P99 286 µs – **đạt** ngưỡng < 5 ms một cách thoải mái và cả < 1 ms.

> **Nếu cần giảm jitter hơn nữa thì làm gì?**

- **PREEMPT_RT kernel:** biến hầu hết IRQ handler thành kernel thread ưu tiên, loại bỏ vùng không thể preempt – đây là cải thiện lớn nhất (kết quả QEMU ở trên còn bị ảnh hưởng bởi nhân PREEMPT_NONE).
- **CPU isolation / cpuset:** cô lập một core cho Sensor Thread (`isolcpus` / cpuset) để không bị các task khác chen vào.
- **IRQ affinity:** gom ngắt (timer, mạng) sang core khác với core chạy sensor; tăng `HZ`/dùng hrtimer (đã bật HIGH_RES_TIMERS + HZ=1000).
- **Giảm công việc trong đường nóng:** tránh `printf` trong Sensor khi verbose, ưu tiên biến atomic thay vì lock, dùng `mlockall` (đã làm) để tránh page fault.

Mục 6: Đo lường bộ nhớ (Task 2)

Chạy `sms_app -i 500 -d 120` trong QEMU, đọc `/proc/PID/status` tại T=5 s, 30 s, 55 s và `/proc/PID/maps`.

Chỉ số	T = 5 s	T = 30 s	T = 55 s	Xu hướng
VmSize	3 648 kB	3 648 kB	3 648 kB	không đổi
VmRSS	2 536 kB	2 536 kB	2 536 kB	**không tăng** (no leak)
VmData	2 128 kB	2 128 kB	2 128 kB	không đổi
Threads	5	5	5	main + 4 worker

RSS phẳng suốt thời gian chạy chứng minh circular buffer (cấp phát 64 slot một lần) và log queue (128 slot cố định) không phình theo thời gian – khớp với kết quả Valgrind 0 byte lost.

```
TASK 2 – Memory footprint T=5/30/55s + /proc/PID/maps
/ # sms_app -i 500 -d 120 & P=$!; sleep 6; echo "=== T=5s ==="; grep -E "VmRSS
|V
mSize|VmStk|VmData|Threads" /proc/$P/status; sleep 24; echo "=== T=30s ==="; g
re
p -E "VmRSS|VmSize|VmStk|VmData|Threads" /proc/$P/status; sleep 25; echo "===
T=
55s ==="; grep -E "VmRSS|VmSize|VmStk|VmData|Threads" /proc/$P/status; echo "=
==
 /proc/PID/maps (35 dong dau) ==="; head -35 /proc/$P/maps; kill -INT $P; slee
p
4; echo "=== /proc/sms_stats sau khi chay ==="; cat /proc/sms_stats
=== Sensor Monitoring System (SMS) – LAB/ASSIGNMENT-02 ===
interval=500 ms  temp_thr=40.0 C  hum_thr=80.0 %  duration=120 s  driver mode
sms_sensor: interval set to 500 ms
[INIT] driver interval set to 500 ms
[INIT] mlockall OK (pages locked, no page-fault jitter)
[INIT] 4 threads running (sensor, processor, alert, logger)
[ALERT] started
[SENSOR] SCHED_FIFO prio 50 : OK
[LOGGER] writing to /tmp/sms.log
[PROCESSOR] started
[ALERT][17629] HUMID_HIGH: VALUE 80.389 THRESHOLD 80.000
[ALERT][18131] TEMP_SPIKE: VALUE -5.646 THRESHOLD 5.000
[ALERT][18631] TEMP_SPIKE: VALUE 5.573 THRESHOLD 5.000
[ALERT][19131] TEMP_SPIKE: VALUE 13.668 THRESHOLD 5.000
[ALERT][19631] TEMP_SPIKE: VALUE -9.734 THRESHOLD 5.000
[ALERT][20131] TEMP_SPIKE: VALUE -9.530 THRESHOLD 5.000
[ALERT][20631] TEMP_SPIKE: VALUE 14.210 THRESHOLD 5.000
[ALERT][22131] TEMP_SPIKE: VALUE -16.940 THRESHOLD 5.000
=== T=5s ===
VmSize:        3648 kB
VmRSS:         2536 kB
VmData:        2128 kB
VmStk:          132 kB
Threads:    5
[ALERT][22631] TEMP_HIGH: VALUE 42.073 THRESHOLD 40.000
[ALERT][22631] TEMP_SPIKE: VALUE 19.809 THRESHOLD 5.000
[ALERT][23131] TEMP_SPIKE: VALUE -17.191 THRESHOLD 5.000
[ALERT][24131] TEMP_SPIKE: VALUE -8.401 THRESHOLD 5.000
[ALERT][24631] TEMP_HIGH: VALUE 41.279 THRESHOLD 40.000
[ALERT][24631] HUMID_HIGH: VALUE 84.355 THRESHOLD 80.000
[ALERT][24631] TEMP_SPIKE: VALUE 19.950 THRESHOLD 5.000
[ALERT][25131] TEMP_SPIKE: VALUE -23.434 THRESHOLD 5.000
[ALERT][25631] TEMP_HIGH: VALUE 43.740 THRESHOLD 40.000
[ALERT][25631] TEMP_SPIKE: VALUE 25.895 THRESHOLD 5.000
```

Hình 10: VmRSS/VmSize tại 3 thời điểm và 5+ vùng trong `/proc/PID/maps`.

Giải thích các vùng trong maps (ứng dụng static nên không có shared libs):

1. `00010000-00083000 r-xp /usr/bin/sms_app` – **text** (mã máy).
2. `00093000-00095000 r--p /usr/bin/sms_app` – **rodata** (hằng, chuỗi).
3. `00095000-00097000 rw-p /usr/bin/sms_app` – **data + bss** (biến toàn cục `g_ctx`).
4. `00097000-00188000 rw-p [heap]` – **heap** (các cấp phát nhỏ, buffer libc).
5. `76e8f000-…-76f92000` – **4 thread stack** (mỗi stack 256 kB + 1 guard page `---p`), tương ứng sensor/processor/alert/logger.
6. `7ea78000-7ea99000 [stack]` – stack của main; `[vdso]/[vvar]/[sigpage]/[vectors]` – vùng nhân hỗ trợ syscall/signal.

Mục 7: Reflection

7.1. Kỹ năng khó nhất và cách vượt qua

Khó nhất là **đồng bộ hóa nhiều thread** và **đảm bảo shutdown sạch**: một thread đang block trên condition variable sẽ không tự biết cờ `stop` đã bật. Ban đầu chương trình treo ở `pthread_join` vì Processor/Logger vẫn nằm trong `pthread_cond_wait`. Tôi giải quyết bằng cách thêm hàm `cb_shutdown()/lq_shutdown()` broadcast cho mọi waiter, và cho Alert Thread dùng `mq_timedreceive` với timeout để định kỳ kiểm tra cờ dừng. Đây là bài học về việc thiết kế "wake-up path" ngay từ đầu chứ không phải thêm sau.

7.2. Nếu làm lại sẽ thiết kế khác gì

Tôi sẽ dùng **eventfd + epoll** hoặc một "shutdown pipe" để đánh thức đồng loạt thay vì broadcast từng cấu trúc, và tách rõ **driver abstraction layer** (một interface `sensor_source`) ngay từ đầu để chế độ `--sim` không cần rẽ nhánh trong vòng lặp. Tôi cũng sẽ thêm **unit test cho circular buffer** (đẩy/pop đồng thời) và dùng `perf` ngay từ khi viết thay vì phát hiện busy-poll muộn.

7.3. Kỹ năng dùng nhiều nhất khi đi làm

Đọc và gỡ lỗi ở tầng hệ thống: dùng `dmesg/strace/perf//proc` để lần ra nguyên nhân thay vì đoán; cùng với **quản lý đồng bộ và vòng đời tài nguyên** trong đa luồng. Đây là những thứ xuất hiện hàng ngày khi phát triển firmware dịch vụ chạy dài.

7.4. Cải tiến muốn thêm cho SMS

Thêm **publish MQTT/HTTP** để đẩy dữ liệu cảm biến ra dashboard, và **lưu cấu hình/ngưỡng xuống JFFS2** trên NAND để giữ cấu hình qua reset (tái dùng kiến thức MTD/JFFS2 của LAB-02). Ngoài ra có thể thêm adaptive sampling: khi phát hiện bất thường thì tự giảm chu kỳ lấy mẫu để bám sát hiện tượng.

> **Kết luận:** Hệ thống chạy 60 s ổn định với 4 thread song song, 119 mẫu, 0 buffer overflow, 120 lần đọc driver (khớp kỳ vọng), shutdown sạch qua SIGINT. Valgrind 0 leak/0 lỗi; jitter SCHED_FIFO P99 286 µs (tốt hơn CFS ~35%); perf phát hiện và tối ưu được hotspot giảm ~240× CPU.